Lab 03 — The life cycle of a container

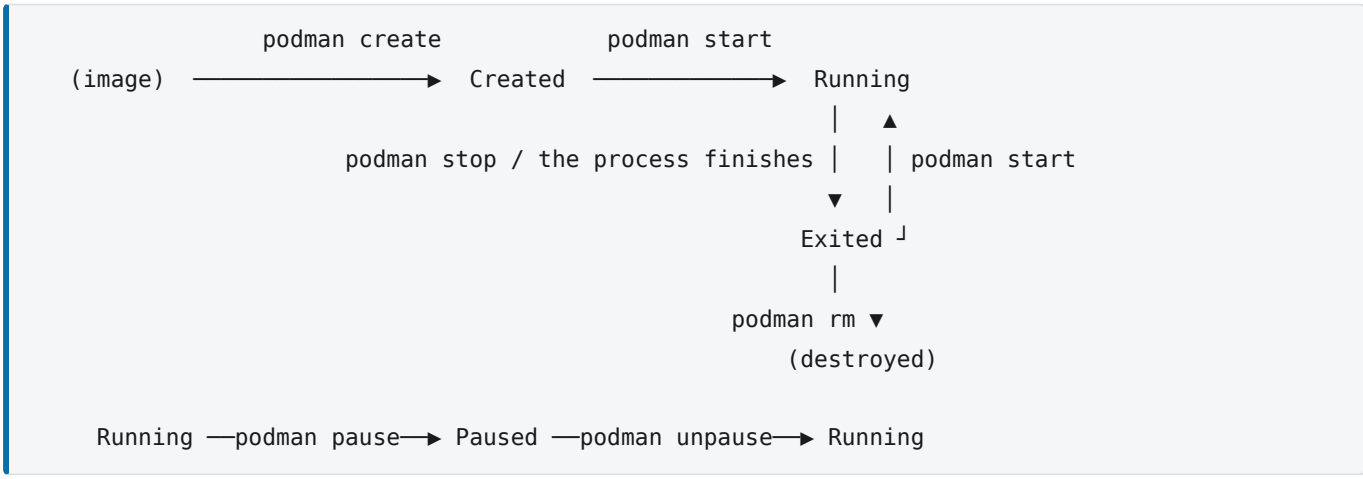
Theory — how a container is born, lives, receives signals, and dies; why PID 1 governs everything, and who restarts your containers when there is no daemon.

Objectives

- Know the states of a container and the commands that move it between them.
- Understand why a container "stops on its own".
- Master the relationship between the **main process**, **signals**, and the **exit code**.
- Choose between `exec` and `attach`, between foreground and background.
- Know what a *restart policy* really does — and what it cannot do without a daemon.

---

1. The states



Command	Effect
`podman create`	Prepares the container (writable layer, config) without starting it
`podman start`	Starts the main process
`podman run`	`create` + `start` (+ `pull` if the image is missing)
`podman stop`	Asks politely for a stop, then forces one after a delay
`podman kill`	Forces the stop immediately
`podman restart`	`stop` followed by `start`
`podman pause`	Freezes the processes (cgroup *freezer*) without stopping them
`podman rm`	Destroys the container **along with its writable layer**

An `Exited` container is not dead: it keeps its configuration, writable layer, and logs. You can inspect it, restart it, or pull files out of it. Only `rm` destroys it.

2. The fundamental rule: a container lives as long as its PID 1

> **REMEMBER**
> A container stops at exactly one moment: when its **main process** ends. Not before, not after. That is the whole rule.

This explains almost every case of "my container stops on its own":

- `podman run alpine` exits immediately — the default command is `/bin/sh`, and with no terminal attached, the shell has nothing to read and exits.
- `podman run nginx` stays alive — nginx runs in the foreground and never exits on its own.
- A script that pushes a service into the **background** (`java -jar … &`) dies immediately: the script finishes, and PID 1 finishes with it.

This leads to a design rule: **an image starts its service in the foreground**. No daemon, no `systemd`, no `nohup` inside a container — the engine itself acts as the service manager.

> **→ LINUX**
> On a regular Linux machine, PID 1 is `init` (nowadays `systemd`): the first process the kernel creates, the ancestor of every other process. The kernel gives it special treatment. If PID 1 dies, the whole system halts. And it **ignores by default any signal** it has not installed a handler for, so a careless `kill` cannot bring the machine down. Inside a container, *your application* inherits that `init` status — privileges and traps included.

Corollary: a container is built for **one main process**. Putting the API and the database in the same container breaks the model — you can no longer restart, monitor, or scale them separately.

3. Foreground, background, and the `-it` duo

```
podman run nginx                # foreground: the terminal is blocked, logs shown
podman run -d nginx             # detached: returns your prompt, prints the container ID
podman run -it alpine sh        # interactive: you get a usable shell
podman run --rm alpine date     # one-off execution, container removed on exit
```

`-it` causes confusion because it is actually **two separate options**. `-i` keeps standard input open; without it, a shell finds its `stdin` closed and exits at once. `-t` allocates a pseudo-terminal: the prompt, key echo, `Ctrl+C`. Use `-i` alone in scripts and CI; use `-it` when a human sits at the keyboard.

> **PITFALL**
> In the foreground, `Ctrl+C` sends `SIGINT` to the container's process and nginx stops. In `-it` mode, press `Ctrl+P` then `Ctrl+Q` to do the opposite: **detach without stopping** the container.

> **PODMAN**
> With Docker, "detached" means the daemon keeps track of the container. Podman has no daemon. When `podman run -d` returns your prompt, **`conmon`** stays behind — a few hundred KB, one per container. It keeps the `stdout` / `stderr` pipes open, writes the logs, and records the exit code when PID 1 dies. You will spot it in `ps` right above your containers. Close your WSL session and `conmon` dies along with your containers… unless `systemd` keeps them alive (section 6).

4. Signals: how a container dies

`podman stop` is not an on/off switch. It follows a protocol:

1. It sends **`SIGTERM`** to PID 1: "shut down cleanly".
2. It waits through a **grace period**, 10 seconds by default (change it with `-t`).
3. If the process is still alive, it sends **`SIGKILL`** — uncatchable and immediate. Podman says so: `StopSignal SIGTERM failed to stop container … in 10 seconds, resorting to SIGKILL`.

`podman kill` jumps straight to step 3. And `podman rm -f` performs a full `stop`, 10 seconds included — which is why lab 01 used `-t 0`.

> **→ LINUX**
> A **signal** is an asynchronous notification the kernel delivers to a process. `SIGTERM` (15) requests a stop and can be caught; `SIGKILL` (9) kills outright and cannot be caught; `SIGINT` (2) is your `Ctrl+C`. A program "handles" a signal by installing a *handler*; otherwise the default action applies — for `SIGTERM`, the process dies. PID 1 is the exception: it has no default action, so it simply ignores the signal.

During those 10 seconds, a well-written Spring Boot application finishes its in-flight requests, closes the PostgreSQL pool, and unregisters from service discovery. `SIGKILL` allows none of that: requests get cut off, connections hang on the database side, and data may end up inconsistent.

> **→ JAVA / SPRING BOOT**
> The JVM turns `SIGTERM` into a run of its **shutdown hooks** (`Runtime.addShutdownHook`). Spring Boot registers one that closes the application context: `@PreDestroy` methods, the JDBC pool, the web server. With `server.shutdown=graceful` (and `spring.lifecycle.timeout-per-shutdown-phase=20s`), the server stops accepting connections and lets in-flight requests finish. All of this works **only if** **`SIGTERM`** **actually reaches** the JVM.

Two traps that keep the signal from arriving:

1. A shell sitting in front of the application. This happens when a start-up script launches the application without `exec`, or when a `CMD` uses the *shell* form (`CMD java -jar app.jar` becomes `/bin/sh -c "java -jar app.jar"`). PID 1 is then `sh`, and `sh` **does not forward** `SIGTERM` to its child. Java never sees the signal, 10 seconds pass, and the process is killed. The fix has two parts: use the *exec* form (`CMD ["java","-jar","app.jar"]`), and in a script, make `exec java -jar app.jar` the last line. Lab 04 covers this syntax detail in depth; it determines whether your containers shut down cleanly.

2. The special status of PID 1. A process that runs as PID 1 without a `SIGTERM` handler is **immune** to `podman stop`; it just gets killed after the delay. PID 1 must also "adopt" orphaned processes, or *zombies* pile up. That is what `--init` is for: it puts a proper mini-init (`podman-init`) in front of your application.

5. Exit codes

```
podman run --rm alpine sh -c 'exit 3'; echo $?     # 3
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
```

A container's exit code is the exit code of its PID 1, kept in its status (`Exited (3)`). Learn to recognise these:

Code	Usual meaning
`0`	Normal termination
`1`	Generic application error
`125`	The engine itself failed (invalid option)
`126`	Command found but not executable (or `pasta` could not open the port)
`127`	Command not found in the image
`137`	Killed by `SIGKILL` (128+9) — `podman kill`, expired grace period, or the **OOM killer**
`143`	Terminated by `SIGTERM` (128+15) — a clean `stop`

`137` is the code you will meet most often in production: a `stop` that overran the grace period, or a memory limit that was exceeded. `podman inspect` settles it: `.State.OOMKilled` is `true` in the memory case.

6. Restart policies — and who enforces them

```
podman run -d --restart=unless-stopped --name api my-api:1.0
```

Policy	Behaviour
`no` (default)	No automatic restart
`on-failure[:N]`	Restarts on a non-zero exit code, at most N times
`always`	Always restarts, including after a host reboot… **if someone is there to do it**
`unless-stopped`	Like `always`, unless you stopped it yourself

With Docker, the daemon enforces these rules, including when the machine boots. With Podman, `conmon` restarts the container for as long as your session lives — but after a reboot, nothing is left to read the policy.

PODMAN
Podman's answer is **systemd**, Linux's service manager, through **Quadlet**: a ten-line file at `~/.config/containers/systemd/api.container` (`[Container]`, `Image=`, `PublishPort=`…) plus `systemctl --user start api`. The container becomes an ordinary service: it starts at boot, restarts on failure, and logs to `journalctl`. This is exactly why Podman never wanted a daemon of its own — Linux already has one. Lab 10 puts this into practice.

PITFALL
`always` restarts a container even after a manual `stop`, the next time the engine starts. `unless-stopped` remembers what you meant: on a single machine, it is almost always the right choice.

7. Observe and intervene

```
podman logs -f --tail 50 api        # output stream of PID 1
podman exec -it api sh              # new process INSIDE the container
podman attach api                   # reconnect to the existing PID 1
podman top api                      # the container's processes, seen from the host
podman stats api                    # CPU/memory consumption in real time
podman inspect api                  # complete state, JSON
podman cp api:/app/log.txt .        # extract a file, even from a stopped container
podman events --since 10m           # the journal of creations, stops, deaths
```

`exec` **creates a new process** in the container's namespaces — use it when you want to look around inside. `attach` reconnects you to the input and output of the **existing PID 1**: press `Ctrl+C` there and the container stops.

`podman logs` shows only what PID 1 wrote to `stdout` / `stderr`, as captured by `conmon`. An application that writes to a file shows nothing here — hence the rule: **log to standard output**. Spring Boot does so by default, so do not configure a `logging.file.name`.

8. In the workplace

- The Spring Boot back end runs with `-d`, with `--restart=unless-stopped` under Docker or as a Quadlet service under Podman — or with no policy at all under an orchestrator, which handles restarts itself.
- Clean shutdown is a production concern: `SIGTERM` delivered + Spring *graceful shutdown* = deployments that lose no requests.
- Diagnosis always follows the same sequence: `ps -a` (status, code), `logs` (what the application said), `inspect` (OOM? configuration?), then `exec` if the container is still alive.

Remember

- A container lives exactly as long as its main process (PID 1).
- Services must run **in the foreground**: no daemon, no `&`, no `systemd` inside the container.
- `-i` keeps `stdin` open, `-t` allocates a terminal. `stop` = `SIGTERM`, grace period, then `SIGKILL` (Podman warns you); `kill` = `SIGKILL` right away; `rm -f` = a full `stop` unless you add `-t 0`.
- The *exec* form (`["java","-jar","x.jar"]`) is essential for receiving signals.
- `137` = killed (KILL or OOM), `143` = stopped cleanly (TERM), `127` = command not found.
- `rm` destroys the container's data; `stop` does not. `exec` creates a process, `attach` hooks onto PID 1. Without a daemon, restarting at boot goes through systemd (Quadlet).

Vocabulary

PID 1: the container's main process. — **grace period**: the time between `SIGTERM` and `SIGKILL`. — **graceful shutdown**: a clean stop that finishes work in progress. — **restart policy**: automatic restart rule. — **OOM killer**: the kernel kills a process when memory runs out. — **zombie**: a terminated process whose exit code nobody read. — **conmon**: the supervisor of a Podman container. — **Quadlet**: Podman's integration with systemd (`.container` files).